SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CLEANING REAL-WORLD FINANCIAL DATA

DSA0503 - QUERY PROCESSING FOR DATA SCIENCE

Presented by:

R YASWANTH REDDY -192424337

NARESH M – 192424381

Supervised by:

Dr.C.Anuradha , PROFESSOR/CSE

INTRODUCTION

- Financial data is widely used in banking, investments, transactions, and business decisions.
- Real-world financial data is often collected from multiple sources.
- This data may contain missing values, duplicate records, and incorrect information.
- It can also have inconsistent formats, spelling errors, and unusual values.
- These problems can affect the accuracy and reliability of financial analysis.
- Data cleaning helps identify and correct these issues.
- The main goal is to create accurate, consistent, and analysis-ready financial data.

Cleaning Using Python

Raw Financial Dataset

Transaction ID	Customer Name	Date	Amount	Category
T001	Arun Kumar	01/05/2026	₹5,000	Food
T002	Priya Sharma	2026-05-02	7500	food
T003	Arun Kumar	03/05/2026	NULL	Shopping
T004	Ravi Kumar	04/05/2026	₹2,500,000	Shopping
T004	Ravi Kumar	04/05/2026	₹2,500,000	Shopping
T005	Priya Sharm	05/05/2026	3500 INR	FOOD
T006	Kiran	NULL	4200	Travel

Code

```
data = {
    "Transaction_ID": ["T001", "T002", "T003", "T004", "T004", "T005", "T006"],
    "Customer_Name": ["Arun Kumar", "Priya Sharma", "Arun  Kumar", "Ravi Kumar",
                      "Ravi Kumar", "Priya Sharm", "Kiran"],
    "Date": ["01/05/2026", "2026-05-02", "03/05/2026", "04/05/2026",
             "04/05/2026", "05/05/2026", None],
    "Amount": ["₹5,000", "7500", None, "₹2,500,000", "₹2,500,000", "3500 INR", "4200"],
    "Category": ["Food", "food", "Shopping", "Shopping", "Shopping", "FOOD", "Travel"]
}

df = pd.DataFrame(data)

print("RAW DATA")
print(df)

print("\nMISSING VALUES")
print(df.isnull().sum())

df = df.drop_duplicates()

df["Amount"] = (
    df["Amount"]
    .astype(str)
    .str.replace("₹", "", regex=False)
    .str.replace(",", "", regex=False)
    .str.replace("INR", "", regex=False)
    .replace("None", np.nan)
)
```

```
df["Amount"] = pd.to_numeric(df["Amount"], errors="coerce")

Q1 = df["Amount"].quantile(0.25)
Q3 = df["Amount"].quantile(0.75)
IQR = Q3 - Q1

lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

df["Outlier"] = (df["Amount"] < lower_limit) | (df["Amount"] > upper_limit)

df["Date"] = pd.to_datetime(df["Date"], dayfirst=True, errors="coerce")
df["Date"] = df["Date"].dt.strftime("%Y-%m-%d")

df["Customer_Name"] = (
    df["Customer_Name"]
    .str.replace(r"\s+", " ", regex=True)
    .str.strip()
)

df["Customer_Name"] = df["Customer_Name"].str.title()

df["Category"] = df["Category"].str.strip().str.title()

df["Transaction_ID_Valid"] = df["Transaction_ID"].apply(
    lambda x: bool(re.fullmatch(r"T\d{3}", x))
)

names = df["Customer_Name"].dropna().unique()

similar_pairs = []
```

Code

```
similar_pairs = []

for i in range(len(names)):
    for j in range(i + 1, len(names)):
        similarity = SequenceMatcher(None, names[i], names[j]).ratio()
        if 0.80 <= similarity < 1.0:
            similar_pairs.append((names[i], names[j], round(similarity, 2)))

print("\nFUZZY MATCHING RESULTS")

for pair in similar_pairs:
    print(pair[0], "<->", pair[1], "Similarity:", pair[2])

print("\nOUTLIERS")

print(df[df["Outlier"]][["Transaction_ID", "Customer_Name", "Amount"]])

print("\nCLEANED DATA")

print(df)
```

Handling Missing Values

print(df.isnull().sum())

- Date has 1 missing value.
- Amount has 1 missing value.
- Missing financial information must be handled carefully.
- We should not blindly guess financial values.

```
MISSING VALUES
Transaction_ID     0
Customer_Name      0
Date               1
Amount             1
Category           0
dtype: int64
```

Removing Duplicate Records

```
print("Before:", len(df))
df = df.drop_duplicates()
print("After:", len(df))
```

```
Before: 6
After: 6
```

Explanation

- The dataset originally contains 7 records.
- One duplicate transaction was found.
- After removing it, 6 unique records remain.
- This prevents double-counting in financial analysis.

Standardizing Amounts

Problem:

Amounts are stored in different formats:

- ₹5,000
- 7500
- ₹2,500,000
- 3500 INR
- 4200

```
df["Amount"] = (
    df["Amount"]
    .astype(str)
    .str.replace("₹", "", regex=False)
    .str.replace(",", "", regex=False)
    .str.replace("INR", "", regex=False)
)

df["Amount"] = pd.to_numeric(df["Amount"], errors="coerce")

print(df["Amount"])
```

```
0       5000.0
1       7500.0
2          NaN
3    2500000.0
5       3500.0
6       4200.0
```

Detecting Outliers

Problem

T004 has an amount of:

₹2,500,000

while most transactions are between ₹3,500 and ₹7,500.

Code:

```
Q1 = df["Amount"].quantile(0.25)
Q3 = df["Amount"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[
    (df["Amount"] < lower) |
    (df["Amount"] > upper)
]
print(outliers[["ID", "Amount"]])
```

```
OUTLIERS
  Transaction_ID Customer_Name     Amount
3           T004    Ravi Kumar  2500000.0
```

Standardizing Dates and Categories

Problem

Dates:

01/05/2026 , 2026-05-02 ,03/05/2026

Categories:

Food, food ,FOOD

Code:

```
df["Date"] = pd.to_datetime(
    df["Date"],
    dayfirst=True,
    errors="coerce"
)

df["Date"] = df["Date"].dt.strftime("%Y-%m-%d")

df["Category"] = df["Category"].str.strip().str.title()

print(df[["Date", "Category"]])
```

	Date	Category
0	2026-05-01	Food
1	2026-05-02	Food
2	2026-05-03	Shopping
3	2026-05-04	Shopping
5	2026-05-05	Food
6	NaN	Travel

Regex and Fuzzy Matching

1. Regex — Pattern Validation

- Regex can check whether structured financial information follows the required format.

	ID	Valid_ID
0	T001	True
1	T002	True
2	T003	True
3	T004	True
5	T005	True
6	T006	True

Code:

```
import re
df["Valid_ID"] = df["ID"].apply(
    lambda x: bool(re.fullmatch(r"T\d{3}", x))
)
print(df[["ID", "Valid_ID"]])
```

Fuzzy Matching

2. Fuzzy Matching

Our dataset contains:

- Priya Sharma
- Priya Sharm

These names are similar but not exactly identical.

Fuzzy matching can identify them as potentially referring to the same customer.

Code:

```
from difflib import SequenceMatcher
a = "Priya Sharma"
b = "Priya Sharm"
similarity = SequenceMatcher(None, a, b).ratio()
print(round(similarity, 2))
```

```
FUZZY MATCHING RESULTS
Priya Sharma <-> Priya Sharm Similarity: 0.96
```

Cleaned Dataset

ID	Customer	Date	Amount	Category
T001	Arun Kumar	2026-05-01	5000	Food
T002	Priya Sharma	2026-05-02	7500	Food
T003	Arun Kumar	2026-05-03	—	Shopping
T004	Ravi Kumar	2026-05-04	2500000	Shopping
T005	Priya Sharm	2026-05-05	3500	Food

Conclusion

- Real-world financial data often contains multiple inconsistencies.
- Python provides efficient techniques for identifying and cleaning these problems.
- Missing values and duplicates must be handled carefully.
- Outliers should be investigated before removal.
- Standardization improves consistency.
- Regex and fuzzy matching help identify hidden data-quality problems.
- The final cleaned dataset provides a reliable foundation for financial analysis and decision-making.


SIMATS ENGINEERING
SIMATS
Saveetha Institute of Medical And Technical Sciences
EANING REAL-WORLD FINANCIAL DATA
DSA0503 - QUERY PROCESSING FOR DATA SCIENCE
192424337
Supervised by:
Dr.C.Anuradha , PROFESSOR/CSE
GPS Map Camera
Mevalurkuppam, Tamil Nadu, India
22g8+c47, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.025936° Long 80.015636°
Thursday, 13/08/2026 09:33 AM GMT +05:30
Google